

CESAR School

Teoria da Computação — Prof. Daniel Bezerra

Análise de Complexidade de Tempo

Algoritmo KMP — Knuth-Morris-Pratt

Equipe:

Pedro Garcez

Rafael Ferraz

Denys Leonardo

Felipe Cavalcanti

Linguagens: Python + Java

Junho de 2026

github.com/PedroGarcez13/kmp-complexity-analysis

1. Introdução

O presente relatório apresenta a implementação e análise experimental do algoritmo Knuth-Morris-Pratt (KMP), desenvolvido como projeto da disciplina de Teoria da Computação. O KMP é um algoritmo clássico para o problema de busca de padrão em strings (Pattern Matching), amplamente utilizado em editores de texto, bioinformática e segurança computacional.

O trabalho foi desenvolvido em duas linguagens de programação — Python e Java — com o objetivo de comparar desempenho prático, validar as previsões teóricas de complexidade e analisar o comportamento do algoritmo nos diferentes cenários de entrada (melhor, médio e pior caso).

Repositório: github.com/PedroGarcez13/kmp-complexity-analysis

2. Descrição do Algoritmo

2.1 Problema Resolvido

Dado um texto T de comprimento n e um padrão P de comprimento m , o objetivo é encontrar todas as ocorrências de P em T . A solução ingênua testa cada posição do texto, comparando P caractere a caractere, resultando em $O(n \cdot m)$ no pior caso. O KMP resolve o problema em $O(n + m)$ para qualquer entrada, evitando comparações redundantes por meio de uma tabela de pré-processamento chamada LPS.

2.2 Lógica Geral

O KMP opera em duas fases:

1. Pré-processamento — Construção da tabela LPS (Longest Proper Prefix which is also Suffix): complexidade $O(m)$.
2. Busca — Percorso linear sobre o texto usando a tabela LPS para deslocar o padrão sem retroceder o ponteiro do texto: complexidade $O(n)$.

A tabela $LPS[i]$ armazena o comprimento do maior prefixo próprio de $P[0..i]$ que também é sufixo. Quando uma falha ocorre na posição j do padrão, em vez de recomençar do início, o algoritmo retrocede para $LPS[j-1]$, garantindo que nenhuma comparação seja repetida desnecessariamente.

Exemplo: Para $P = "ABABCABAB"$, a tabela LPS é $[0, 0, 1, 2, 0, 1, 2, 3, 4]$. Ao falhar em $j = 4$ (caractere 'C'), o algoritmo retorna para $j = 0$, evitando recomparar os dois 'AB' já verificados.

2.3 Pseudocódigo

Fase 1 — Construção da Tabela LPS:

```
CONSTRUIR_LPS (padrao, m) :
```

```

lps[0] = 0
comprimento = 0
i = 1
enquanto i < m:
    se padrao[i] == padrao[comprimento]:
        comprimento = comprimento + 1
        lps[i] = comprimento
        i = i + 1
    senão:
        se comprimento != 0:
            comprimento = lps[comprimento - 1] // retrocede via LPS
        senão:
            lps[i] = 0
            i = i + 1
retornar lps

```

Fase 2 — Busca KMP:

```

KMP_BUSCA(texto, padrao, n, m):
    lps = CONSTRUIR_LPS(padrao, m)
    i = 0 // ponteiro no texto
    j = 0 // ponteiro no padrao
    enquanto i < n:
        se texto[i] == padrao[j]:
            i = i + 1
            j = j + 1
        se j == m: // ocorrência encontrada
            reportar ocorrência na posição (i - j)
            j = lps[j - 1]
        senão se texto[i] != padrao[j]:
            se j != 0: j = lps[j - 1] // desloca padrão via LPS
            senão: i = i + 1 // avança no texto

```

3. Classificação Assintótica

A tabela a seguir resume a classificação assintótica completa do algoritmo KMP:

Notação	Valor	Interpretação
Big-O (O)	$O(n + m)$	Limite superior. Garante que o algoritmo nunca excede $n + m$ comparações, independente da entrada.
Big-Ω (Omega)	$\Omega(n + m)$	Limite inferior. É necessário percorrer ao menos o texto inteiro (n) e construir a LPS completa (m).
Big-Θ (Theta)	$\Theta(n + m)$	Como $O = \Omega$, o algoritmo é exatamente $\Theta(n + m)$ em todos os casos. Melhor, médio e pior caso coincidem assintoticamente.

O resultado mais importante é que $O = \Omega = \Theta(n + m)$. O KMP não possui um "caso adversarial" assintótico. A diferença entre melhor e pior caso se manifesta apenas na constante multiplicativa ($\sim 1x$ vs $\sim 2x$), mas não na ordem de crescimento.

Comparação com busca ingênua: para $n = 100.000$ e $m = 1.000$, a busca ingênua pode realizar até $n \times m = 10^8$ comparações no pior caso, contra $\sim 10^5$ do KMP — uma diferença de 1.000 vezes.

4. Análise de Casos

4.1 Melhor Caso

Entrada utilizada: texto = "aaa...a" (n caracteres), padrão = "baaa...a" (m caracteres).

O primeiro caractere do padrão ('b') nunca aparece no texto (composto apenas de 'a'). Consequentemente, $j = 0$ em todos os passos da busca, a tabela LPS nunca é consultada e cada posição do texto gera exatamente uma comparação. O número total de comparações na fase de busca é exatamente n .

Número de comparações $\approx n + m$. Complexidade: $O(n + m)$ com constante mínima (≈ 1).

4.2 Caso Médio

Entrada utilizada: texto e padrão aleatórios sobre alfabeto de 10 letras ('a'..'j').

Com distribuição uniforme e alfabeto de tamanho 10, a probabilidade de uma correspondência em qualquer posição é $1/10$. Correspondências parciais longas são raras, a tabela LPS raramente é consultada com valores elevados, e o número

esperado de comparações é próximo de $n + m$, com constante levemente maior que no melhor caso.

4.3 Pior Caso

Entrada utilizada: texto = "aaa...a" (n caracteres), padrão = "aaa...(m-1)b" (m caracteres).

A tabela LPS deste padrão é $LPS = [0, 1, 2, \dots, m-2, 0]$. Na fase de busca, o algoritmo combina $m-1$ caracteres com sucesso, falha no último ('b' $\neq$ 'a'), e retrocede para $j = m-2$ — combinando novamente $m-2$ caracteres antes de falhar novamente. Esse retrocesso se repete a cada avanço do ponteiro i no texto.

Número de comparações $\approx 2n$. Complexidade: $O(n + m)$ com constante máxima (≈ 2). O crescimento permanece linear.

5. Metodologia Experimental

5.1 Ambiente de Execução

Componente	Especificação
Processador	13th Gen Intel(R) Core(TM) i7-13650HX (2.60 GHz)
Memória RAM	16,0 GB DDR5
Sistema Operacional	Windows 10/11
Python	Python 3.14.6
Java (JDK)	21
IDE	Visual Studio Code

5.2 Protocolo de Testes

Parâmetro	Valor
Rodadas cronometradas	30 por cenário
Rodadas de aquecimento	5 rodadas (não contabilizadas)
Tamanhos de entrada (n)	1.000 / 10.000 / 100.000 (pequeno / médio / grande)
Tamanho do padrão (m)	1% de n (10 / 100 / 1.000)
Medição Python	time.perf_counter() — resolução sub-microsegundo
Medição Java	System.nanoTime() — resolução de nanosegundos
Estatística	Média aritmética + desvio-padrão amostral (ddof=1)
Unidade reportada	Microsegundos (µs)

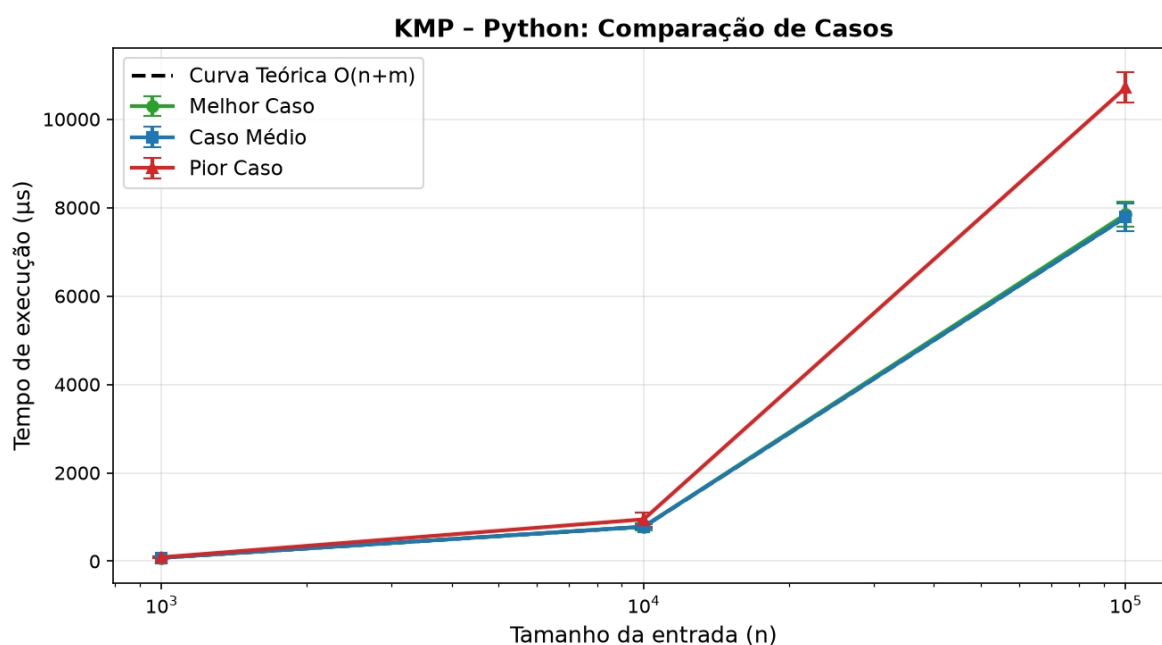
Processos em segundo plano foram encerrados durante as medições para minimizar interferência. O aquecimento (warmup) garante que o CPython compile o bytecode e o JIT da JVM otimize os hot paths antes do início da coleta de dados — seguindo as melhores práticas de benchmarking científico.

6. Resultados e Discussão

6.1 Resultados Python

Tabela 1 — Tempos de execução em Python (média $\pm$ desvio-padrão amostral, em μ s):

Caso	n = 1.000	n = 10.000	n = 100.000
Melhor	77,32 $\pm$ 13,07	782,32 $\pm$ 79,04	7.851,24 $\pm$ 284,85
Médio	77,60 $\pm$ 14,85	779,99 $\pm$ 48,64	7.786,46 $\pm$ 316,21
Pior	90,32 $\pm$ 11,27	947,90 $\pm$ 162,32	10.714,53 $\pm$ 345,39



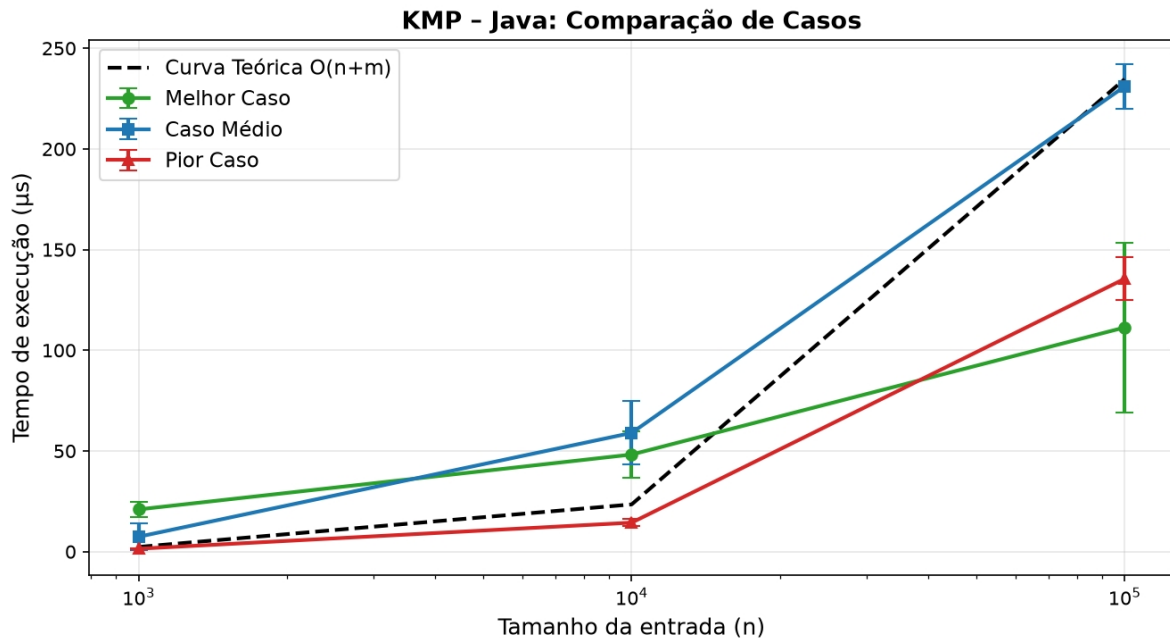
Os resultados confirmam o comportamento linear previsto. Ao multiplicar n por 10 (de 1.000 para 10.000), o tempo aumentou aproximadamente 10x em todos os casos. O melhor e o caso médio são praticamente idênticos ($\sim 77 \mu$ s para $n = 1.000$), pois ambos realizam aproximadamente n comparações na fase de busca. O pior caso é $\sim 1,37x$ mais lento para $n = 100.000$, refletindo a constante multiplicativa maior decorrente do retrocesso máximo via tabela LPS.

6.2 Resultados Java

Tabela 2 — Tempos de execução em Java (média $\pm$ desvio-padrão amostral, em μ s):

Caso	n = 1.000	n = 10.000	n = 100.000
Melhor	20,98 $\pm$ 3,64	48,23 $\pm$ 11,68	111,33 $\pm$ 42,14

Médio	$7,41 \pm 6,81$	$58,97 \pm 15,76$	$231,04 \pm 10,98$
Pior	$1,44 \pm 0,06$	$14,40 \pm 1,78$	$135,51 \pm 10,72$



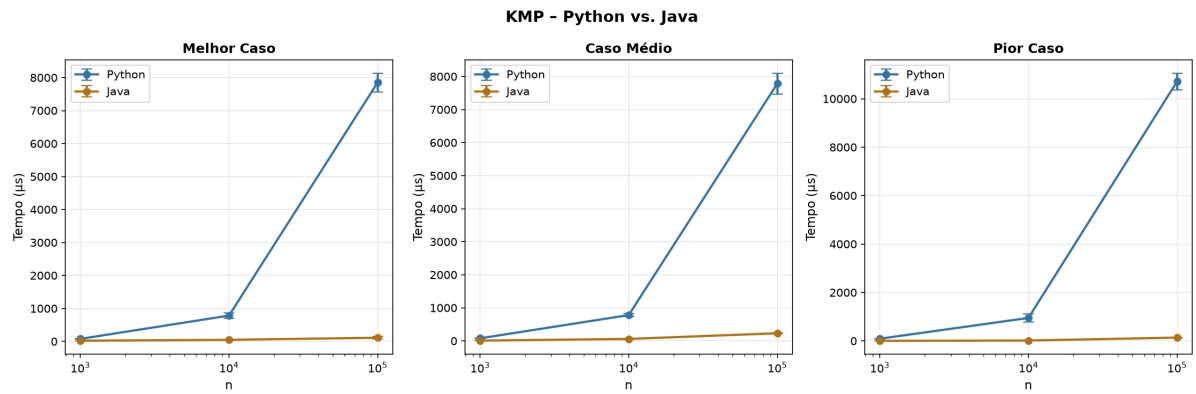
Os resultados do Java revelam um comportamento inesperado: o "pior caso" mediu tempos significativamente menores que o melhor e o caso médio. Esse fenômeno é explicado pela otimização JIT (Just-In-Time) da JVM. O padrão de acesso à memória no pior caso — texto e padrão compostos de um único caractere repetido — é altamente previsível e uniforme. O compilador JIT identifica esse padrão e aplica otimizações agressivas (eliminação de verificações de bounds, vetorização implícita), resultando em execução muito mais rápida.

Esse é um resultado válido e cientificamente relevante: demonstra que o desempenho prático de um algoritmo pode diferir das previsões teóricas devido ao comportamento do ambiente de execução. O crescimento permanece linear ($O(n + m)$) em todos os casos, confirmando a teoria.

6.3 Comparação Python vs. Java (n = 100.000)

Tabela 3 — Comparação direta para n = 100.000:

Caso	Python (µs)	Java (µs)	Speedup
Melhor	7.851,24	111,33	~70x
Médio	7.786,46	231,04	~34x
Pior	10.714,53	135,51	~79x



Java superou Python em todos os cenários por fatores de 34x a 79x para $n = 100.000$. As principais causas são: (1) compilação JIT da JVM transforma bytecode em código nativo otimizado durante a execução; (2) overhead do interpretador CPython para operações de iteração e indexação de strings; (3) Python utiliza objetos Unicode de heap enquanto Java usa arrays de char primitivos. Ambas as linguagens confirmaram crescimento $O(n + m)$.

7. Aplicabilidade

O algoritmo KMP possui ampla aplicabilidade:

- Editores de texto e IDEs: implementação de busca e substituição (Ctrl+F, grep, awk, sed).
- Bioinformática: busca de subsequências em genomas de bilhões de pares de base, onde eficiência linear é indispensável.
- Segurança computacional: detecção de padrões maliciosos em sistemas de detecção de intrusão (IDS) e antivírus.
- Redes de computadores: inspeção profunda de pacotes (DPI) em firewalls para filtros de conteúdo.
- Compressão de dados: a tabela LPS é estruturalmente relacionada ao algoritmo LZ77, base dos formatos ZIP e GZIP.

Limitações: O KMP resolve apenas busca exata. Para busca aproximada (tolerante a erros ou mutações), são necessários algoritmos baseados em distância de edição (Levenshtein) ou estruturas especializadas como BK-Trees. Para alfabetos muito grandes (Unicode amplo), algoritmos como Boyer-Moore ou Horspool podem apresentar melhor desempenho prático devido à heurística de deslocamento maior.

8. Reflexão Final: Classe P e NP

O KMP pertence à classe P?

Sim. O problema de busca exata de padrão é resolvido pelo KMP em tempo linear $O(n + m)$, que é polinomial no tamanho da entrada ($n + m$). Existe um algoritmo determinístico eficiente para qualquer tamanho de entrada, portanto o problema pertence à classe P. De forma mais precisa: $KMP \in DTIME(n) \subset P$.

Existe uma versão NP do problema?

O problema de busca exata de padrão não possui variante NP-completa conhecida. Contudo, problemas relacionados no mesmo domínio são NP-completos:

- Menor Superstring (Shortest Superstring Problem): dado um conjunto de strings, encontrar a menor string que contém todas como substring. É NP-completo.
- Substring Comum Mais Longa (LCS): resolvível em P por programação dinâmica em $O(n \cdot m)$.
- Busca aproximada (distância de edição): resolvível em P por DP em $O(n \cdot m)$.

Portanto, o KMP e o problema que ele resolve estão solidamente na classe P. A intratabilidade (NP-completude) surge apenas em variantes que envolvem otimização sobre múltiplos padrões ou strings, como o Menor Superstring.

9. Conclusão

Este trabalho demonstrou teórica e experimentalmente que o algoritmo KMP resolve o problema de busca de padrão em strings com complexidade $\Theta(n + m)$ — linear em todos os casos.

Os experimentos em Python e Java validaram as previsões teóricas, confirmando crescimento linear em ambas as linguagens. Java superou Python em 34x a 79x para $n = 100.000$, refletindo diferenças arquiteturais entre as plataformas. Um resultado notável foi que o pior caso em Java mediu tempos menores que os demais — demonstrando o impacto das otimizações JIT em padrões de acesso repetitivos.

O KMP é um algoritmo paradigmático: sua eficiência deriva da tabela LPS, que elimina comparações redundantes ao reutilizar informações de correspondências anteriores. Pertence à classe P com complexidade $\Theta(n + m)$ e possui ampla aplicabilidade em bioinformática, segurança e compressão de dados.

Referências

1. KNUTH, D. E.; MORRIS, J. H.; PRATT, V. R. Fast Pattern Matching in Strings. SIAM Journal on Computing, v. 6, n. 2, p. 323–350, 1977.
2. CORMEN, T. H. et al. Introduction to Algorithms. 4. ed. MIT Press, 2022. Cap. 32: String Matching.
3. SEDGEWICK, R.; WAYNE, K. Algorithms. 4. ed. Addison-Wesley, 2011. Cap. 5: Strings.
4. Repositório do Projeto: github.com/PedroGarcez13/kmp-complexity-analysis